

Compose 활용

16 부수효과 (1)

부수효과

16.

부수효과 (1)

LaunchedEffect: 컴포저블의 범위에서 정지 함수 실행

rememberCoroutineScope: 컴포지션 인식 범위를 확보하여 컴포저블 외부에서 코루틴 실행

rememberUpdatedState: 값이 변경되는 경우 다시 시작되지 않아야 하는 효과에서 값 참조

DisposableEffect: 정리가 필요한 효과

SideEffect: Compose 상태를 비 Compose 코드에 게시

produceState: 비 Compose 상태를 Compose 상태로 변환

derivedStateOf: 하나 이상의 상태 객체를 다른 상태로 변환

snapshotFlow: Compose의 상태를 Flow로 변환

LaunchedEffect: 컴포저블의 범위에서 정지 함수 실행

16.

부수효과 (1)

```
@Composable
fun MyScreen(
    state: UiState<List<Movie>>,
    scaffoldState: ScaffoldState = rememberScaffoldState()
) {

    if (state.hasError) {

        LaunchedEffect(scaffoldState.snackbarHostState) {
            scaffoldState.snackbarHostState.showSnackbar(
                message = "Error message",
                actionLabel = "Retry message"
            )
        }
    }

    Scaffold(scaffoldState = scaffoldState) {
    }
}
```


LaunchedEffect: 컴포저블의 범위에서 정지 함수 실행

16.

부수효과 (1)

```
LaunchedEffect(scaffoldState.snackbarHostState) {
    scaffoldState.snackbarHostState.showSnackbar(
        message = "Error message",
        actionLabel = "Retry message"
    )
}
```

Suspend 함수를 실행할 수 있음.

key가 바뀌면 다시 코루틴을 다시 만들어 수행.

(scaffoldState.snackbarHostState)

LaunchedEffect가 콤포지션에서 빠지면 코루틴 캔슬.

rememberCoroutineScope: 컴포지션 인식 범위를 확보하여 컴포저블 외부에서 코루틴 실행

16.

부수효과 (1)

```
@Composable
fun MoviesScreen(scaffoldState: ScaffoldState =
rememberScaffoldState()) {
    val scope = rememberCoroutineScope()
    Scaffold(scaffoldState = scaffoldState) {
        Column {
            Button(
                onClick = {
                    scope.launch {
                        scaffoldState.snackbarHostState.showSnackbar("Something
happened!")
                    }
                }
            ) {
                Text("Press me")
            }
        }
    }
}
```

rememberCoroutineScope: 컴포지션 인식 범위를 확보하여 컴포저블 외부에서 코루틴 실행

16.
부수효과 (1)

```
@Composable
fun MoviesScreen(scaffoldState: ScaffoldState
= rememberScaffoldState()) {
    val scope = rememberCoroutineScope()
    ...
}
```

MovieScreen 범위에서 코루틴 스코프를 만듦.

rememberUpdatedState: 값이 변경되는 경우 다시 시작되지 않아야 하는 효과에서 값 참조

16.
부수효과 (1)

```
@Composable
fun LandingScreen(onTimeout: () -> Unit) {

    val currentOnTimeout by rememberUpdatedState(onTimeout)

    LaunchedEffect(true) {
        delay(SplashWaitTimeMillis)
        currentOnTimeout()
    }
}
```

rememberUpdatedState: 값이 변경되는 경우 다시 시작되지 않아야 하는 효과에서 값 참조

16.

부수효과 (1)

```
@Composable
fun <T> rememberUpdatedState(newValue: T): State<T> = remember {
    mutableStateOf(newValue)
}.apply { value = newValue }
```

값을 상태로 만들고 remember 한 후, 다시 apply로 값을 대입

remember에서는 상태가 만들어지면 리컴포지션 과정에 예전 값을 가지게 됨.

그래서 값을 다시 apply로 설정하는 것.

rememberUpdatedState: 값이 변경되는 경우 다시 시작되지 않아야 하는 효과에서 값 참조

16.
부수효과 (1)

```
@Composable
fun LandingScreen(onTimeout: () -> Unit) {

    val currentOnTimeout by rememberUpdatedState(onTimeout)

    LaunchedEffect(true) {
        delay(SplashWaitTimeMillis)
        currentOnTimeout()
    }
}
```

DisposableEffect: 정리가 필요한 효과

16.

부수효과 (1)

```
@Composable
fun HomeScreen(
    lifecycleOwner: LifecycleOwner = LocalLifecycleOwner.current,
    onStart: () -> Unit,
    onStop: () -> Unit
) {
    val currentOnStart by rememberUpdatedState(onStart)
    val currentOnStop by rememberUpdatedState(onStop)

    DisposableEffect(lifecycleOwner) {
        val observer = LifecycleEventObserver { _, event ->
            if (event == Lifecycle.Event.ON_START) {
                currentOnStart()
            } else if (event == Lifecycle.Event.ON_STOP) {
                currentOnStop()
            }
        }

        lifecycleOwner.lifecycle.addObserver(observer)

        onDispose {
            lifecycleOwner.lifecycle.removeObserver(observer)
        }
    }
}
```

DisposableEffect: 정리가 필요한 효과

16.
부수효과 (1)

```
DisposableEffect(lifecycleOwner) {
    ...

    onDispose {
        lifecycleOwner.lifecycle.removeObserver(observer)
    }
}
```

Key가 바뀌면 dispose하고 이펙트를 재시작.

컴포지션에서 이펙트가 나가도 onDispose 호출

리소스 등을 관리할 때도 사용 가능.

SideEffect: Compose 상태를 비 Compose 코드에 게시

16.
부수효과 (1)

```
@Composable
fun rememberAnalytics(user: User): FirebaseAnalytics {
    val analytics: FirebaseAnalytics = remember {
        /* ... */
    }

    SideEffect {
        analytics.setUserProperty("userType", user.userType)
    }
    return analytics
}
```

SideEffect는 매 Recomposition마다 호출 됨.

produceState: 비 Compose 상태를 Compose 상태로 변환

```
@Composable
fun loadNetworkImage(
    url: String,
    imageRepository: ImageRepository
): State<Result<Image>> {

    return produceState<Result<Image>>(initialValue = Result.Loading, url, imageRepository) {
        val image = imageRepository.load(url) // suspend call
        value = if (image == null) {
            Result.Error
        } else {
            Result.Success(image)
        }
    }
}
```

produceState는 코루틴을 제공

State를 리턴 (State<Result<Image>>)

value 값을 통해 상태를 반환

derivedStateOf: 하나 이상의 상태 객체를 다른 상태로 변환

```
@Composable
fun TodoList(highPriorityKeywords: List<String> = listOf("Review", "Unblock", "Compose")) {

    val todoTasks = remember { mutableStateListOf<String>() }

    val highPriorityTasks by remember(highPriorityKeywords) {
        derivedStateOf { todoTasks.filter { it.containsWord(highPriorityKeywords) } }
    }

    Box(Modifier.fillMaxSize()) {
        LazyColumn {
            items(highPriorityTasks) { /* ... */ }
            items(todoTasks) { /* ... */ }
        }
    }
}
```

derivedStateOf는 State를 만듦.

연결된 바뀌면 계산이 다시 수행될 수 있음.

계산된 값이 바뀔 때만 State의 value가 바뀜.

highPriorityKeywords는 상태가 아니기 때문에 remember에 키로.

snapshotFlow: Compose의 상태를 Flow로 변환

16.

부수효과 (1)

```
val listState = rememberLazyListState()

LazyColumn(state = listState) {
    // ...
}

LaunchedEffect(listState) {
    snapshotFlow { listState.firstVisibleItemIndex }
        .map { index -> index > 0 }
        .distinctUntilChanged()
        .filter { it == true }
        .collect {
            MyAnalyticsService.sendScrolledPastFirstItemEvent()
        }
}
```

snapshotFlow는 State<T>를 콜드 Flow로 변경.

이전에 방출한 값(emitted value)과 다를 경우에 방출(emit).